Engineering Certified JSON Derivers for Rocq with ELPI

Will Thomas and Perry Alexander

University of Kansas, Lawrence, USA
{30wthomas, palexand}@ku.edu

Abstract. Practical verification produces extracted code, and extracted code eventually crosses system boundaries. JSON is a common boundary format for configuration, test data, counterexamples, traces, and tool pipelines, but serializers at this boundary are often hand-written and unverified. We present an ELPI-based deriver for JSON that automatically generates `Jsonifiable` instances for a practical fragment of Rocq inductive and record types. Each generated instance contains an encoder, a decoder, and a Rocq-checked proof that decoding the encoded value recovers the original. The contribution is a performance-conscious tool for extending the verified trust perimeter of extracted Rocq components to their JSON boundary. The tool is distributed as the `rocq-json` opam package.

1 Introduction

Automated verification is often evaluated by the proof it produces, but practical formal methods also has to run. A verified component may be extracted to OCaml, linked into a test harness, used as a runtime monitor, embedded in a larger tool chain, or deployed as part of a verified system. As soon as that component communicates with the outside world, the proof assistant's internal data representation crosses a boundary.

JSON is a common format at that boundary. Model-checker front-ends exchange structured system descriptions and counterexamples, verified runtime monitors consume event logs, test generators exchange inputs and results, and verified protocols serialize messages for unverified peers. The problem is that the serialization layer is often outside the trusted development. Extraction preserves the behavior of Rocq [8] terms, but it does not automatically certify the hand-written encoder or decoder that maps those terms to a deployment format. A bug in that code can invalidate the deployed verification story.

As a concrete example, consider a verification pipeline with two extracted Rocq components: a verified analyzer that emits counterexample witnesses and a verified replay checker that independently validates those witnesses. The components may be developed, extracted, and deployed separately, with JSON as the wire format between them. If the shared boundary type is a trace datatype containing event records, branch choices, and final states, then the JSON format

is part of the proof boundary: a hand-written encoder that drops a constructor, or a decoder that interprets fields in a different order, can make the replay checker validate a value different from the analyzer's internal witness. The individual Rocq proofs still hold, but the composition of the two verified tools is mediated by unverified boundary code. A shared `Jsonifiable` instance turns that boundary into a checked contract. The `canonical_jsonification` theorem says precisely that the receiving decoder for the agreed JSON representation recovers the value emitted by the sender's encoder, which is the property needed when verified tools exchange verification artifacts through an ordinary system format.

`rocq-json` addresses this gap by extending the formal trust perimeter to the JSON boundary. It provides the following typeclass:

```
Class Jsonifiable (A : Type) := {
  to_JSON    : A → JSON;
  from_JSON  : JSON → Result A string;
  canonical_jsonification : forall (a : A), from_JSON (to_JSON a) = res a
}.
```

The deriver described here generates all three components for many ordinary inductive and record types. It is implemented in `coq-elpi` and exposed through the command `Elpi derive.jsonifiable T` [7]. For verification-oriented applications that produce JSON, the central guarantee is exactly the displayed one: a value that leaves the verified component through the encoder can be decoded back to the same Rocq value.

This is a tool paper about practical verification engineering with code that may cross assurance boundaries. The substantially new contribution is the integration of ELPI inspection of Rocq declarations, generated code that remains suitable for extraction, typeclass parameters for nested serializable fields, automatic proof construction for the round-trip theorem, and an artifact that measures both applicability and extracted performance. The same ingredients are broadly useful for other derivers that construct computational content and accompanying correctness proofs.

Availability. The tool is available as the `rocq-json` opam package. The public development repository and review artifact are available at `https://github.com/ku-sldg/rocq-json`.

2 Tool Overview

The user-facing interface is deliberately small. Given a declaration such as an enumeration, algebraic data type, parameterized container, or record, the user invokes the deriver and receives a registered typeclass instance. Nullary constructors are encoded as JSON strings. Constructors with arguments are encoded as singleton JSON objects whose key is the constructor name and whose value

stores named fields.[1] Records are encoded as JSON objects keyed by projection names.

For example, the following declaration and command are enough to produce an encoder, decoder, proof of *invertibility*, and registered typeclass instance:

```
Inductive UserRole : Type :=
| Admin  | Guest  | Moderator (level : nat)
| StandardUser (id : nat) (username : string).
Elpi derive.jsonifiable UserRole.
```

The generated representation uses a string for nullary constructors and an object for constructors with fields:

```
Example role_roundtrip (r : UserRole) :
  from_JSON (to_JSON r) = res r := canonical_jsonification r.
Example moderator_json :
  to_JSON (Moderator 2) = JSON_Object [
    ("Moderator", JSON_Object [("level", JSON_Nat 2)])] := eq_refl.
```

For parameterized types, the generated instance takes serializers for type parameters. For recursive types, the generated encoder and decoder use structurally recursive definitions. The current implementation also supports a conservative collection of nested recursive occurrences through standard data containers such as lists, options, and products. More general nested or mutually recursive types are currently reported as unsupported rather than handled partially.[2]

```
Inductive BinTree (A : Type) : Type :=
| BinLeaf
| BinNode (left : BinTree A) (value : A) (right : BinTree A).
Elpi derive.jsonifiable BinTree.
Check BinTree_Jsonifiable : ∀ A, Jsonifiable A → Jsonifiable (BinTree A).
```

Failures are reported as ordinary Rocq errors whose message names the blocking shape when it is recognized. For example, attempting to derive an instance for `sig` fails with:

```
Error:
derive.jsonifiable: dependent constructor fields are not supported by
Jsonifiable. Field: ... type head: unknown type constructor
```

The benchmark script used in Section 4 also acts as a batch diagnostic pass over installed Corelib/Stdlib sources: it records the failure category, source location, generated probe file, and complete Rocq log for every candidate.

[1] For anonymous arguments, the deriver uses the first character of the argument type's head constructor name as a base key. Duplicate base keys are disambiguated with numeric suffixes, so two anonymous `nat` fields become `n1` and `n2`.

[2] The issue of generating quality `to/from_JSON` functions is not as difficult as proving them correct without quality structural recursion procedures. This may be mitigated as better support for nested inductives in Rocq is actively being pursued [3]

The generated proof is part of the public contract. A successful derivation does not merely produce functions with plausible behavior; it registers a `canonical_jsonification` proof. This property is intentionally one-sided. It says that values produced by the encoder can be decoded back to the original value. For extracted verified components that emit JSON for other tools or services, this is the direction that protects the boundary: the serialized message has not lost or changed the Rocq value. The converse property, that every accepted JSON value is a canonical representation, is a different contract. It would require specifying a canonical JSON grammar for each type and proving that the decoder rejects all non-canonical alternatives, which is useful but substantially more representation-specific.

3 Implementation

The deriver follows the shape of the Rocq declaration. ELPI is an embedded λProlog language used by `coq-elpi` to inspect and construct Rocq terms at the meta-level. The deriver first reads the inductive or record view, classifies parameters and constructor arguments, and then builds separate skeletons for the encoder, decoder, proof, and final typeclass instance. The skeletons are elaborated by Rocq before being added as constants. This staging has been useful in practice: ELPI performs structural analysis and code generation, while Rocq remains responsible for checking the terms that become part of the trusted development.

Two engineering choices are central. First, field serializers are kept abstract through typeclass arguments where possible. This makes generated instances compose with existing hand-written or derived instances, including standard containers. In practice, this means that although `nat` in Rocq is inductively defined as `O | S _`, we are better served by making it an actual number (e.g. 0, 1, 42). Second, decoder generation is performance-sensitive. Large enumerations are decoded using a string decision tree that extracts to direct pattern matching over characters, avoiding a long chain of string equality tests. The resulting extracted code is close to the shape a careful and performance oriented user would write by hand.

The proof-generation strategy is mixed. General recursive and record cases use Rocq proof automation specialized to the generated encoders and decoders. Pure enumerations use a direct generated proof term, avoiding proof-search overhead on a common stress case. This division is not theoretically deep, but it matters for predictable tool latency.

4 Evaluation

We evaluate two questions: how broadly the deriver applies to existing Rocq declarations, and whether generated extracted code is competitive with hand-written code on stress cases.

4.1 Applicability

The artifact includes a benchmark script that scans installed Corelib and Stdlib sources for inductive-like declarations, probes each candidate in isolation, records diagnostics, and then compiles a single benchmark file containing all successful derivations. This workflow is deliberately conservative: unsupported declarations remain visible as categorized failures. Declarations that are visible in source but are not closed derivation targets, such as declarations inside functor bodies or module signatures, are recorded separately rather than discarded.

Table 1. Corelib/Stdlib applicability and failure categories.

Metric	Count
Discovered declarations	342
Recorded, not probed	31
Probed declarations	311
Successful probes	81
Probe failures	230
Probe timeouts	0
Timed successful derivations	81

Failure category	Count
prop-target-not-supported	150
dependent-field-not-supported	48
indexed-family-not-supported	21
function-field-not-supported	4
proof-field-not-supported	3
sort-field-not-supported	3
coinductive-not-supported	1

The successful probes include 50 inductives, 17 variants, and 14 records or structures; 39 are from Corelib and 42 are from Stdlib.

The category names in Table 1 are a triage device, not a claim that every failed probe is a tool defect. Some declarations are clear non-targets for serialization: propositions are erased by extraction, functions have no canonical JSON representation, sort-valued fields contain types rather than values, and coinductive values may be infinite. The genuine current limitations are concentrated around dependent data, where decoding must reconstruct values whose types depend on earlier fields or external indices. Table 2 gives one representative declaration for each category and the rationale used by the artifact.

Table 2. Representative failure examples and classification rationale.

Category and example	Rationale
`prop-target-not-supported: or`	The target lives in `Prop`; propositions are erased at extraction and are not runtime data.
`dependent-field-not-supported: sig`	A later field depends on an earlier witness; decoding must reconstruct a value and its dependent type.
`indexed-family-not-supported: reflect`	The declaration is indexed by values; decoding requires recovering the index or fixing it in the instance.
`function-field-not-supported: implies`	Arbitrary functions have no canonical data representation in JSON or ordinary serializers.
`proof-field-not-supported: sumbool`	A computational wrapper carries proof evidence; decoding requires proof erasure or reconstruction.
`sort-field-not-supported: Tlist`	Constructor fields contain sorts such as `Type`; these are types, not JSON values.
`coinductive-not-supported: Stream`	Potentially infinite data needs an explicit depth-bounded or lazy serialization contract.

The important distinction is between non-targets and genuine engineering limitations. The non-target group is not a design choice particular to `rocq-json`: there is simply no runtime proposition to serialize after extraction; arbitrary functions do not have a canonical finite data representation; sorts such as `Type` and `Set` are not ordinary values; and coinductive data needs a separate bounded or lazy contract. These account for 158 probe failures. The remaining large categories are dependent fields and indexed families, 69 cases in total. Those are real future work: the JSON decoder would have to recover enough information to reconstruct a dependent value, or the deriver would need a fixed-index or sigma-style encoding. The three proof-field cases sit on the same boundary: the outer type is computational, but successful decoding would require erasing or rebuilding proof evidence that extraction normally removes. The 31 recorded-but-not-probed declarations are reported in the artifact as well: 24 occur inside parameterized modules or functor bodies, such as `Stdlib.FSets.FMapAVL.Raw.tree`, and 7 occur inside module types or signatures, such as `MSetGenTree.Ops.tree`. These are genuine source declarations, but not closed global references to which the deriver can be applied directly *outside the functor*. All categories are reported explicitly so the success rate is clear and auditable.

The raw success rate is therefore intentionally not the main applicability claim. Of the 342 discovered declarations, the 31 functor/signature declarations are not closed deriver inputs, and 158 probe failures are non-targets in the sense above. The meaningful remaining gap is the dependent fragment: dependent fields, indexed families, and proof-bearing computational wrappers. This is also the fragment where extraction itself erases part of the proof context, so serializing the "whole" dependent value is not always the right deployment contract. The artifact reports these cases explicitly rather than folding them into an undifferentiated failure count. Excluding the 31 functor/signature declarations that are not valid deriver inputs and the 158 non-target declarations, the tool successfully derives instances for 81 of 153 semantically eligible declarations, roughly 53% coverage on the well-defined target fragment, with the remaining gap concentrated in dependent and indexed types.

4.2 Derivation Time

The combined successful benchmark records Rocq's `Time` output for each derived instance. In the reported run, the combined benchmark compiled successfully with wall-clock time 5.21 seconds; the sum of Rocq-reported derivation transactions was 4.29 seconds. The mean derivation time was 0.053 seconds, the median was 0.034 seconds, the 90th percentile was 0.112 seconds, and the maximum was 0.521 seconds.

The slowest success is `byte`, a 256-constructor Corelib enumeration. It is the broad benchmark's version of the large enum stress case used while engineering the string decision tree. The remaining cost is dominated by Rocq-side derivation and proof construction. After extraction, the corresponding dispatch path is close to the handwritten Rocq baseline below.

Table 3. Slowest successful derivations in the broad benchmark.

Declaration	Kind	Seconds
Corelib.Init.Byte.byte	Inductive	0.521
Stdlib.rtauto.Rtauto.proof	Inductive	0.182
Stdlib.btauto.Reflect.formula	Inductive	0.174
Stdlib.micromega.ZMicromega.ZArithProof	Inductive	0.166
Corelib.Floats.FloatClass.float_class	Variant	0.142
Stdlib.micromega.RingMicromega.Psatz	Inductive	0.137
Stdlib.Sorting.Heap.Tree	Inductive	0.125
Stdlib.setoid_ring.Field_theory.FExpr	Inductive	0.116

4.3 Extracted Code Performance

The extraction benchmark compares generated definitions against handwritten Rocq definitions after both have been extracted to OCaml. This keeps the comparison in the same source language and extraction pipeline. The benchmark covers several shapes: a 256-constructor enumeration, a tuple-like constructor, a flat record, two mixed sums with nullary and field-bearing constructors, and a recursive binary tree. Each benchmark was run 10 times. The enum benchmark serializes and deserializes every constructor; the other benchmarks run many iterations over representative values.

Table 4. Extracted round-trip benchmarks.

Benchmark	Generated mean (ms)	Handwritten mean (ms)	Change (%)
enum256	55.843	55.628	+0.4%
point2d	1.282	1.378	-7.0%
userrole	2.316	3.285	-29.5%
serverconfig	0.749	1.960	-61.8%
instruction	2.213	3.491	-36.6%
bintree	4.180	6.416	-34.9%

Negative changes in Table 4 mean the generated code is faster than the handwritten baseline. These benchmarks do not predict every serializer workload, but instead validate the main engineering concern: generated instances should extract to code with the same general performance profile as direct handwritten Rocq definitions. The handwritten baselines were written by the authors in Rocq and extracted through the same pipeline; they were not independently optimized. The comparison should therefore be read as evidence that generated code avoids obvious extraction-time performance regressions, not as a claim of superiority over all possible handwritten serializers or OCaml-specific deriving systems.

5 Related Work and Positioning

Rocq has several metaprogramming routes for deriving boilerplate. MetaCoq [6] provides quoted syntax and verified metaprogramming; `coq-elpi` provides an embedded λProlog language integrated with Rocq elaboration and an existing derive framework. `rocq-json` uses `coq-elpi` because the task is structural inspection plus term construction, with the generated terms checked by Rocq. MetaCoq could in principle support a more deeply verified metaprogram through the SafeTemplateMonad; the tradeoff here is pragmatic. The trusted result is not the ELPI program itself, but the generated Rocq constants and the `canonical_jsonification` theorem that Rocq accepts. This matches the engineering goal of producing executable boundary code and a checked contract with low user overhead. In particular, the use of ELPI allows efficient generation of the `Jsonifiable` instances, as well as hooking into the general "deriviation" framework.

`coq-elpi` already includes a derive framework and examples of type-directed derivers for logical or structural boilerplate. Compared with typical derivers for equality, ordering, showing, decidability, or logical instances, `rocq-json` generates two executable programs and a theorem relating them. That proof obligation changes the problem: it is not enough to inspect constructors and print code, because the generated encoder and decoder must be accepted together with an invertibility proof.

This work is also distinct from Rocq/JSON infrastructure such as SerAPI [1]. SerAPI serializes Rocq's own syntax and protocol data for interaction with external clients; `rocq-json` derives verified `Jsonifiable` instances for user-defined Rocq data types.

Lean provides JSON datatypes and typeclass-based derivation facilities in its programming environment [5, 4] through the `deriving` system. These ecosystems support practical serialization workflows, but the serialization layer is not normally accompanied by a proof, at the source proof-assistant level, that decoding the encoded value recovers the original user datatype.

OCaml libraries such as `ppx_yojson_conv` generate efficient JSON code for production datatypes [2]. These systems are practical and often highly optimized, but they critically do not produce a Rocq-checked theorem tied to the source inductive or record declaration. The substantially new combination here is generated encoder, generated decoder, checked encode-decode theorem, extraction-conscious code shape, and a reproducible artifact that reports both successes and limitations.

6 Limitations and Future Work

The current deriver targets computational data in `Type` or `Set`. It deliberately excludes shapes where serialization is not a well-defined request: propositions are not runtime values after extraction, functions have no canonical finite data representation, sort-valued fields contain types rather than values, and coinductive values require a separate bounded or lazy contract. These exclusions are

not merely conservative engineering choices; they follow from the operational meaning of the values.

The remaining limitations are mostly dependent-type limitations. The deriver does not yet support dependent families, indexed families, or records whose field types depend on earlier fields. Some nested recursion through standard containers is supported, but arbitrary nested and mutually recursive types remain outside the current design.

The JSON representation is also intentionally modest. The paper evaluates the *derivation method and the certified round-trip property*, not complete adherence to the JSON RFC or interoperability with every external JSON parser.

The main extensions are indexed families and proof-bearing data. Indexed families may require fixed-index instances or dependent-pair decoding; proof fields likely require erasure plus user-supplied reconstruction lemmas. More generally, support for dependent data should make the extraction boundary explicit: when proofs or indices are erased at runtime, the right JSON format may be a projection of the computational content rather than a serialization of the full source-level dependent value.

7 Conclusion

`rocq-json`'s ELPI deriver demonstrates that boundary code for extracted verified components can be treated as part of the formal artifact. The generated instances combine executable encoders and decoders with Rocq-checked invertibility proofs, apply across a meaningful fragment of existing library declarations, and extract to competitive code on the evaluated examples. The broader lesson is that certified derivation tools can be utilized in verification efforts to generate performant code *without* leaving the assurance boundary.

References

1. Gallego Arias, E.J.: SerAPI: Machine-Friendly, Data-Centric Serialization for Coq. Tech. rep., MINES ParisTech (Oct 2016), `https://hal-mines-paristech.archives-ouvertes.fr/hal-01384408`
2. Jane Street: ppx_yojson_conv. `https://github.com/janestreet/ppx_yojson_conv`, accessed 2026-05-05
3. Lamiaux, T., Forster, Y., Sozeau, M., Tabareau, N.: Nested Inductive Types (2025), `https://hal.science/hal-05366368`, working paper or preprint
4. Lean developers: Lean 4 JSON support. `https://lean-lang.org/doc/api/Lean/Data/Json/Basic.html`, accessed 2026-05-05
5. de Moura, L., Kong, S., Avigad, J., van Doorn, F., von Raumer, J.: The lean theorem prover (system description). In: Felty, A.P., Middeldorp, A. (eds.) Automated Deduction - CADE-25. pp. 378–388. Springer International Publishing, Cham (2015)
6. Sozeau, M., Anand, A., Boulier, S., Cohen, C., Forster, Y., Kunze, F., Malecha, G., Tabareau, N., Winterhalter, T.: The MetaCoq Project. Journal of Automated Reasoning (Feb 2020). https://doi.org/10.1007/s10817-019-09540-0, `https://inria.hal.science/hal-02167423`

7. Tassi, E.: Elpi: rule-based meta-language for Rocq. In: CoqPL 2025 - The Eleventh International Workshop on Coq for Programming Languages. Denver, United States (Jan 2025), https://inria.hal.science/hal-04990628
8. The Rocq Development Team: The Rocq Prover. Zenodo, Version 9.0.0, 10.5281/zenodo.11551307 (Apr 2025). https://doi.org/10.5281/zenodo.15149629

A Round-Trip Benchmark Rocq Code

The extracted-code benchmarks in Table 4 are defined in the extraction benchmark source. The generated cases use `Elpi derive.jsonifiable`; the primed instances below are the handwritten Rocq baselines extracted and timed by the same harness.

A.1 Benchmark Types

```
(* 1.3. Single Constructor, Multiple Arguments (Tuple-like) *)
Inductive Point2D : Type :=
| MkPoint2D (x : nat) (y : nat).
Elpi derive.jsonifiable Point2D.

(* 1.4. Mixed Arity Sum Type *)
Inductive UserRole : Type :=
| Admin
| Moderator (level : nat)
| StandardUser (id : nat) (username : string)
| Guest.
Elpi derive.jsonifiable UserRole.

(* 1.5. Flat Record with primitive types *)
Record ServerConfig : Type := {
  host : string;
  port : nat;
  use_ssl : bool
}.
Elpi derive.jsonifiable ServerConfig.
```

```
(* 1.9. Flat Instruction Set (Mix of no-args and primitive args) *)
Inductive Instruction : Type :=
| INop
| IPush (val : nat)
| IPop
| ILoad (reg_idx : nat)
| IStore (reg_idx : nat)
| IHalt.
Elpi derive.jsonifiable Instruction.
```

```
Inductive BinTree (A : Type) : Type :=
| BinLeaf
| BinNode (left : BinTree A) (value : A) (right : BinTree A).
Elpi derive.jsonifiable BinTree.
```

```
Inductive enum_256 :=
| e00 | e01 | e02 | e03 | e04 | e05 | e06 | e07
| e08 | e09 | e0A | e0B | e0C | e0D | e0E | e0F
| e10 | e11 | e12 | e13 | e14 | e15 | e16 | e17
| e18 | e19 | e1A | e1B | e1C | e1D | e1E | e1F
| e20 | e21 | e22 | e23 | e24 | e25 | e26 | e27
| e28 | e29 | e2A | e2B | e2C | e2D | e2E | e2F
| e30 | e31 | e32 | e33 | e34 | e35 | e36 | e37
| e38 | e39 | e3A | e3B | e3C | e3D | e3E | e3F
| e40 | e41 | e42 | e43 | e44 | e45 | e46 | e47
| e48 | e49 | e4A | e4B | e4C | e4D | e4E | e4F
| e50 | e51 | e52 | e53 | e54 | e55 | e56 | e57
| e58 | e59 | e5A | e5B | e5C | e5D | e5E | e5F
| e60 | e61 | e62 | e63 | e64 | e65 |  e66 | e67
|  e68 | e69 | e6A | e6B | e6C | e6D | e6E | e6F
|  e70 | e71 | e72 | e73 | e74 | e75 | e76 | e77
|  e78 | e79 | e7A | e7B  |  e7C  |  e7D  |  e7E  |  e7F
| e80 | e81 | e82 | e83 | e84 | e85 | e86 | e87
| e88 | e89 | e8A | e8B | e8C | e8D | e8E | e8F
```

```
| e90 | e91 | e92 | e93 | e94 | e95 | e96 | e97
| e98 | e99 | e9A | e9B | e9C | e9D | e9E | e9F
| eA0 | eA1 | eA2 | eA3 | eA4 | eA5 | eA6 | eA7
| eA8 | eA9 | eAA | eAB | eAC | eAD | eAE | eAF
| eB0 | eB1 | eB2 | eB3 | eB4 | eB5 | eB6 | eB7
| eB8 | eB9 | eBA | eBB | eBC | eBD | eBE | eBF
| eC0 | eC1 | eC2 | eC3 | eC4 | eC5 | eC6 | eC7
| eC8 | eC9 | eCA | eCB | eCC | eCD | eCE | eCF
| eD0 | eD1 | eD2 | eD3 | eD4 | eD5 | eD6 | eD7
| eD8 | eD9 | eDA | eDB | eDC | eDD | eDE | eDF
| eE0 | eE1 | eE2 | eE3 | eE4 | eE5 | eE6 | eE7
| eE8 | eE9 | eEA | eEB | eEC | eED | eEE | eEF
| eF0 | eF1 | eF2 | eF3 | eF4 | eF5 | eF6 | eF7
| eF8 | eF9 | eFA | eFB | eFC | eFD | eFE | eFF.
Time Elpi derive.jsonifiable enum_256.
```

A.2 Handwritten Rocq Baselines

```
Definition Point2D_Jsonifiable' `{JN : Jsonifiable nat} : Jsonifiable Point2D.
refine (Build_Jsonifiable _ (fun p ⇒
  match p with
  | MkPoint2D x y ⇒
      JSON_Object [("MkPoint2D", JSON_Object [("x", to_JSON x); ("y", to_JSON y)])]
  end) (fun js ⇒
  match js with
  | JSON_Object [("MkPoint2D", JSON_Object [("x", xv); ("y", yv)])] ⇒
      x ← from_JSON xv ;;
      y ← from_JSON yv ;;
      res (MkPoint2D x y)
  | _ ⇒ err err_str_json_unrecognized_constructor
  end) _).
destruct a; cbn.
erewrite canonical_jsonification.
erewrite canonical_jsonification.
reflexivity.
Defined.

Definition UserRole_Jsonifiable' `{JN : Jsonifiable nat, JS : Jsonifiable string} :
     Jsonifiable UserRole.
refine (Build_Jsonifiable _ (fun r ⇒
  match r with
  | Admin ⇒ JSON_String "Admin"
  | Moderator level ⇒
      JSON_Object [("Moderator", JSON_Object [("level", to_JSON level)])]
  | StandardUser id username ⇒
      JSON_Object [("StandardUser", JSON_Object [("id", to_JSON id); ("username", to_JSON
     username)])]
  | Guest ⇒ JSON_String "Guest"
  end) (fun js ⇒
  match js with
  | JSON_String "Admin" ⇒ res Admin
  | JSON_String "Guest" ⇒ res Guest
  | JSON_Object [("Moderator", JSON_Object [("level", jlevel)])] ⇒
      level ← from_JSON jlevel ;;
      res (Moderator level)
  | JSON_Object [("StandardUser", JSON_Object [("id", jid); ("username", usernamev)])] ⇒
      id ← from_JSON jid ;;
      username ← from_JSON usernamev ;;
      res (StandardUser id username)
  | _ ⇒ err err_str_json_unrecognized_constructor
  end) _).
destruct a; cbn;
repeat (erewrite canonical_jsonification);
try reflexivity.
Defined.
```

```
Definition ServerConfig_Jsonifiable'
    `{JS : Jsonifiable string, JN : Jsonifiable nat, JB : Jsonifiable bool}
    : Jsonifiable ServerConfig.
refine (Build_Jsonifiable _ (fun cfg ⇒
  match cfg with
  | {| host := h; port := p; use_ssl := ssl |} ⇒
      JSON_Object [("host", to_JSON h); ("port", to_JSON p); ("use_ssl", to_JSON ssl)]
  end) (fun js ⇒
  match js with
  | JSON_Object [("host", jhost); ("port", jport); ("use_ssl", jssl)] ⇒
      h ← from_JSON jhost ;;
      p ← from_JSON jport ;;
      ssl ← from_JSON jssl ;;
      res {| host := h; port := p; use_ssl := ssl |}
  | _ ⇒ err err_str_json_unrecognized_constructor
  end) _).
destruct a; cbn.
repeat (erewrite canonical_jsonification).
reflexivity.
Defined.

Definition Instruction_Jsonifiable' `{JN : Jsonifiable nat} : Jsonifiable Instruction.
refine (Build_Jsonifiable _ (fun i ⇒
  match i with
  | INop ⇒ JSON_String "INop"
  | IPush val ⇒ JSON_Object [("IPush", JSON_Object [("val", to_JSON val)])]
  | IPop ⇒ JSON_String "IPop"
  | ILoad reg_idx ⇒ JSON_Object [("ILoad", JSON_Object [("reg_idx", to_JSON reg_idx)])]
  | IStore reg_idx ⇒ JSON_Object [("IStore", JSON_Object [("reg_idx", to_JSON reg_idx)])]
  | IHalt ⇒ JSON_String "IHalt"
  end) (fun js ⇒
  match js with
  | JSON_String "INop" ⇒ res INop
  | JSON_String "IPop" ⇒ res IPop
  | JSON_String "IHalt" ⇒ res IHalt
  | JSON_Object [("IPush", JSON_Object [("val", jval)])] ⇒
      val ← from_JSON jval ;;
      res (IPush val)
  | JSON_Object [("ILoad", JSON_Object [("reg_idx", jreg_idx)])] ⇒
      reg_idx ← from_JSON jreg_idx ;;
      res (ILoad reg_idx)
  | JSON_Object [("IStore", JSON_Object [("reg_idx", jreg_idx)])] ⇒
      reg_idx ← from_JSON jreg_idx ;;
      res (IStore reg_idx)
  | _ ⇒ err err_str_json_unrecognized_constructor
  end) _).
destruct a; cbn;
repeat (erewrite canonical_jsonification);
try reflexivity.
Defined.

Fixpoint bintree_to_JSON' {A : Type} `{Jsonifiable A} (t : BinTree A) : JSON :=
  match t with
  | BinLeaf _ ⇒ JSON_String "BinLeaf"
  | BinNode _ l value r ⇒
      JSON_Object [("BinNode", JSON_Object [
        ("left", bintree_to_JSON' l);
        ("value", to_JSON value);
        ("right", bintree_to_JSON' r)])]
  end.

Fixpoint bintree_from_JSON' {A : Type} `{Jsonifiable A} (js : JSON) : Result (BinTree A)
     string :=
  match js with
  | JSON_String "BinLeaf" ⇒ res (·BinLeaf A)
  | JSON_Object [("BinNode", JSON_Object [("left", jleft); ("value", jvalue); ("right",
     jright)])] ⇒
      left ← bintree_from_JSON' jleft ;;
```

```
      value ← from_JSON jvalue ;;
      right ← bintree_from_JSON' jright ;;
      res (·BinNode A left value right)
  | _ ⇒ err err_str_json_unrecognized_constructor
  end.

Definition BinTree_Jsonifiable' {A : Type} `{Jsonifiable A} : Jsonifiable (BinTree A).
refine (Build_Jsonifiable _ bintree_to_JSON' bintree_from_JSON' _).
induction a; cbn.
- reflexivity.
- rewrite IHa1.
  rewrite canonical_jsonification.
  rewrite IHa2.
  reflexivity.
Defined.
```

```
Definition enum_256_Jsonifiable' : Jsonifiable enum_256.
Time refine (Build_Jsonifiable _ (fun e ⇒ match e with
  | e00 ⇒ JSON_String "e00" | e01 ⇒ JSON_String "e01"
  | e02 ⇒ JSON_String "e02" | e03 ⇒ JSON_String "e03"
  | e04 ⇒ JSON_String "e04" | e05 ⇒ JSON_String "e05"
  | e06 ⇒ JSON_String "e06" | e07 ⇒ JSON_String "e07"
  | e08 ⇒ JSON_String "e08" | e09 ⇒ JSON_String "e09"
  | e0A ⇒ JSON_String "e0A" | e0B ⇒ JSON_String "e0B"
  | e0C ⇒ JSON_String "e0C" | e0D ⇒ JSON_String "e0D"
  | e0E ⇒ JSON_String "e0E" | e0F ⇒ JSON_String "e0F"
  | e10 ⇒ JSON_String "e10" | e11 ⇒ JSON_String "e11"
```

⋮

```
  | eFE ⇒ JSON_String "eFE" | eFF ⇒ JSON_String "eFF"
  end) (fun e ⇒
    match e with
  | JSON_String "e00" ⇒ res e00 | JSON_String "e01" ⇒ res e01
  | JSON_String "e02" ⇒ res e02 | JSON_String "e03" ⇒ res e03
  | JSON_String "e04" ⇒ res e04 | JSON_String "e05" ⇒ res e05
  | JSON_String "e06" ⇒ res e06 | JSON_String "e07" ⇒ res e07
  | JSON_String "e08" ⇒ res e08 | JSON_String "e09" ⇒ res e09
  | JSON_String "e0A" ⇒ res e0A | JSON_String "e0B" ⇒ res e0B
  | JSON_String "e0C" ⇒ res e0C | JSON_String "e0D" ⇒ res e0D
  | JSON_String "e0E" ⇒ res e0E | JSON_String "e0F" ⇒ res e0F
  | JSON_String "e10" ⇒ res e10 | JSON_String "e11" ⇒ res e11
```

⋮

```
  | JSON_String "eFE" ⇒ res eFE | JSON_String "eFF" ⇒ res eFF
  | _ ⇒ err err_str_json_unrecognized_constructor
  end
  ) _); destruct a; reflexivity.
Qed.
```

A.3 Extraction Command

```
From Corelib Require Import Extraction.

Extraction "jsonifiable_suite.ml"
  enum_256_Jsonifiable enum_256_Jsonifiable'
  Point2D_Jsonifiable Point2D_Jsonifiable'
  UserRole_Jsonifiable UserRole_Jsonifiable'
  ServerConfig_Jsonifiable ServerConfig_Jsonifiable'
  Instruction_Jsonifiable Instruction_Jsonifiable'
  BinTree_Jsonifiable BinTree_Jsonifiable'.
```